Aran Learn 1: Python and Tools

For: everyone, read this first. **Time:** about 6 hours including practice. **You need:** a Windows laptop and an internet connection. No prior coding is assumed.

By the end of this doc you will be able to set up the project on your own laptop, read every file in `research/`, write a small detector with a test, and send your work to the team through GitHub.

1. The tools and why each one exists

Before installing anything, understand what each tool does. Every one of them solves a problem a team of four would otherwise hit on day one.

Tool	What it is	The problem it solves for Aran
Terminal (PowerShell or Git Bash)	A window where you type commands instead of clicking	Every tool below is run by typing a command, and commands can be copied into docs so everyone gets the same result
Python 3.11 or newer	The programming language all of Aran's research code is written in	Easy to read, and has free libraries for cryptography, maths and machine learning
pip	Python's package installer	Downloads libraries such as `cryptography` and `numpy` so we do not write AES ourselves
Virtual environment (venv)	A private folder of libraries for one project	Stops Aran's library versions from clashing with your other projects
git	A version-control tool that records every change	Four people can edit the same files without overwriting each other, and any mistake can be undone
GitHub	A website that hosts git repositories	The shared copy of the code that everyone pulls from and pushes to
VS Code	A free code editor	Colours the code, shows errors as you type, and has a built-in terminal
Obsidian	A Markdown note app	The repo folder `D:\Aran` doubles as an Obsidian vault, so these docs open as linked notes

A mental model. Think of the repo as a shared notebook. git is the rule that every page change is signed and dated. GitHub is the cupboard where the master copy lives. Your laptop holds a photocopy; you `pull` to refresh your photocopy and `push` to send your pages back.

2. Setup on Windows, step by step

Do these steps in order. After each step there is a check command; do not move on until the check prints what is shown.

Step 1: Install Python. Download the Windows installer from python.org. On the first screen, tick **"Add python.exe to PATH"** before clicking Install. PATH is the list of folders Windows

searches when you type a command; without this tick, typing `python` gives "not recognised".

```
> python --version
Python 3.12.4
```

Step 2: Install git. Download from git-scm.com and accept the defaults. It also installs **Git Bash**, a terminal that understands Linux-style commands such as `ls` and `bash`, which our `fetch_data.sh` script needs.

```
> git --version
git version 2.46.0.windows.1
```

Then tell git who you are, once. These names appear on every commit you make.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Step 3: Install VS Code from code.visualstudio.com, open it, go to Extensions (Ctrl+Shift+X) and install "Python" by Microsoft.

Step 4: Get the code.

```
cd D:\
git clone https://github.com/itzz-inbathamizhans/aran.git Aran
cd Aran
git checkout claude/project-thread-9hgid0
```

`clone` copies the whole repo with its full history. The `checkout` line switches to the branch where the research code currently lives; once that branch is merged into `main` you can skip it.

Step 5: Create the virtual environment. Run this inside `D:\Aran`.

```
python -m venv .venv
.venv\Scripts\activate
```

Your prompt now starts with `(.venv)`. That prefix means "libraries I install go into this project only". You must activate it again every time you open a new terminal. In Git Bash the activate command is `source .venv/Scripts/activate`.

Step 6: Install the libraries.

```
pip install cryptography numpy scikit-learn sklearn-crfsuite pytest
```

Library	Used for	Doc that teaches it
`cryptography`	AES, the building block of FF1 encryption	Learn 3
`numpy`	Fast maths on arrays of numbers	Learn 4 and 6
`scikit-learn`	Machine learning models and metrics	Learn 5
`sklearn-crfsuite`	The CRF model for finding names	Learn 5
`pytest`	Running tests	Section 5 of this doc

Step 7: Run the research code. Open Git Bash in `D:\Aran\research`.

```
bash fetch_data.sh               # downloads 32,000 real public log lines
python -m pytest test_core.py    # every test should say PASSED
python experiment.py             # about 3 minutes; rewrites results.json
```

When something goes wrong.

Error message	Cause	Fix
`'python' is not recognized`	PATH tick was missed	Re-run the Python installer, choose Modify, tick "Add to PATH"
`ModuleNotFoundError: No module named 'cryptography'`	venv not active, or library not installed	Run `.venv\Scripts\activate`, then the `pip install` line
`running scripts is disabled on this system`	PowerShell blocks activate scripts	Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once
`bash: fetch_data.sh: No such file`	You are in the wrong folder	`cd D:\Aran\research` first; `ls` shows the files in the folder
`fatal: not a git repository`	You ran git outside `D:\Aran`	`cd D:\Aran`

3. git for a team of four

The core ideas

- **Commit:** a saved snapshot of the files with a message. Like a save point in a game; you can always return to it.
- **Branch:** a separate line of commits. `main` is the team's agreed version. You work on your own branch so half-finished work never breaks `main`.
- **Pull request (PR):** a request on GitHub to merge your branch into `main`. A teammate reads the changes first. This is where bugs get caught.
- **Merge conflict:** two people changed the same lines. git cannot guess which is right, so it asks you.

Daily routine

When	Command	What it does
Start of the day	`git checkout main` then `git pull`	Get everyone's latest work
Before new work	`git checkout -b arun/otp-classifier`	Make your own branch, named `yourname/topic`
While working	`git status`	See which files you changed
After a small step works	`git add otp.py test_otp.py`	Choose the files for the next snapshot
	`git commit -m "Add OTP context classifier"`	Save the snapshot with a clear message
End of the day	`git push -u origin arun/otp-classifier`	Upload your branch to GitHub
Work is ready	Open a pull request on GitHub	Ask a teammate to review

A good commit message says what changed in the imperative: "Add Luhn check to card detector", not "changes" or "fixed stuff". Small commits are easier to review and easier to undo.

Worked example: resolving a merge conflict

Aasif and Bharadhwaj both edit the same line of `config.py`. When Bharadhwaj pulls, git writes both versions into the file:

```
<<<<<<< HEAD
THRESHOLD = 0.9
=======
THRESHOLD = 0.95
>>>>>>> main
```

The part above `=======` is his version; the part below is what came from `main`. He talks to Aasif, they agree on 0.95, he deletes the marker lines so only `THRESHOLD = 0.95` remains, then runs `git add config.py` and `git commit`. Conflicts are normal, not a disaster.

4. The Python you need for Aran

Aran's code uses a small set of Python ideas again and again. Each one below has an example taken from the kind of data Aran handles. Type each example into a file and run it; reading alone is not enough.

4.1 Variables, types and printing

A variable is a name for a value. Python has a few basic types.

```
aadhaar = "234123412346"     # str (text). Identifiers are text, not numbers.
length = len(aadhaar)        # int (whole number): 12
score = 0.97                 # float (decimal)
is_valid = True              # bool (True or False)
print(f"{aadhaar} has {length} digits, score {score}")
# 234123412346 has 12 digits, score 0.97
```

Why identifiers are text, not numbers. A card number like `0012...` would lose its leading zeros as an `int`, and `int("2341 2341 2346")` fails because of the spaces. Aran always keeps identifiers as strings and converts single digits when it needs to do maths. (The Aadhaar-shaped numbers in these docs are invented and pass the checksum only because they were generated that way.)

The `f"..."` string is an f-string: anything inside `{}` is replaced by its value.

4.2 Strings: slicing, cleaning and masking

Python counts positions from 0. A negative position counts from the end.

```
s = "234123412346"
s[0]       # '2'            first character
s[-1]      # '6'            last character (the check digit)
s[:11]     # '23412341234'  first 11 characters (the payload)
s[-4:]     # '2346'         last 4 characters

raw = "2341-2341 2346"
clean = raw.replace("-", "").replace(" ", "")   # '234123412346'
masked = "XXXX XXXX " + clean[-4:]              # 'XXXX XXXX 2346'
```

4.3 Lists and list comprehensions

A list holds many values in order. A list comprehension builds a list in one line.

```
digits = [int(ch) for ch in "234123412346"]
# [2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 6]
total = sum(digits)            # 35
reversed_digits = digits[::-1] # read right to left, as Luhn and Verhoeff do
```

Read `[int(ch) for ch in s]` as "for each character `ch` in `s`, make `int(ch)`, and collect the results".

4.4 Loops and conditions

`for` repeats work for each item. `enumerate` gives you the position as well. `if` chooses.

```
for i, ch in enumerate(reversed("4111")):
    if i % 2 == 1:
        print(i, ch, "double this one")
    else:
        print(i, ch, "keep as is")
# 0 1 keep as is
# 1 1 double this one
# 2 1 keep as is
# 3 4 double this one
```

`i % 2` is the remainder after dividing by 2, so it is 1 for every odd position. This exact pattern is the heart of the Luhn check in Learn 2.

4.5 Functions

A function packages a step so it can be reused and tested. Every check in Aran is a small function that takes text and returns a value.

```
        def mask_last4(value: str) -> str:
            """Show only the last 4 characters."""
            return "X" * (len(value) - 4) + value[-4:]

        print(mask_last4("234123412346"))   # XXXXXXXX2346
```

The `: str` and `-> str` parts are type hints. Python does not enforce them, but they tell the reader what goes in and what comes out. The text in triple quotes is a docstring that explains the function.

4.6 Dictionaries for counting and lookups

A dict maps keys to values. The experiment uses one to count how many hits each detector finds.

```
        counts = {"aadhaar_regex": 0, "aadhaar_verhoeff": 0}
        counts["aadhaar_regex"] += 1
        counts.get("card_luhn", 0)     # 0: .get returns a default if the key is missing

        prefixes = {"4": "Visa", "60": "RuPay", "65": "RuPay"}
        prefixes["60"]                 # 'RuPay'
```

4.7 Regular expressions

A regular expression (regex) is a search pattern for text. It is the first filter in every leak detector: it finds things with the right shape.

Symbol	Meaning	Example	Matches
`\d`	any digit	`\d\d`	42
`{n}`	exactly n times	`\d{4}`	2341
`{a,b}`	between a and b times	`\d{4,8}`	482913
`[2-9]`	one character from a range	`[6-9]\d{9}`	9876543210
`[ -]?`	an optional space or hyphen	`\d{4}[ -]?\d{4}`	2341 2341, 2341-2341, 23412341
`[A-Z]`	one capital letter	`[A-Z]{5}\d{4}[A-Z]`	ABCPD1234E
`\b`	a word boundary	`\b\d{6}\b`	482913 but not inside 1482913
`( )`	a group you can pull out	`otp is (\d{6})`	captures 482913
`|`	or	`otp|passcode`	either word

Worked example: the Aadhaar pattern from the research code, run on two real-looking lines.

```
import re
AADHAAR = re.compile(r"[2-9]\d{3}[ -]?\d{4}[ -]?\d{4}")

AADHAAR.findall("user kyc ok uid=2341 2341 2346 status=200")
# ['2341 2341 2346']                    a true hit

AADHAAR.findall("blk_4532110988710042 size 9000 3312 4410 ok")
# ['453211098871', '9000 3312 4410']    two false alarms
```

The second line is the whole reason Aran exists. A regex only checks shape, so a Hadoop block ID and three unrelated numbers both look like Aadhaar. On the real logs plain regex gave 366 Aadhaar false alarms; adding the checksum in Learn 2 cut that to 4.

The `r` before the quotes makes a raw string, so `\d` reaches the regex engine unchanged. Always write regexes as raw strings. Practise patterns at regex101.com with the Python flavour selected; it explains every symbol.

4.8 Reading files line by line

Logs are big, so read them one line at a time instead of loading the whole file.

```
hits = 0
with open("data/HDFS_2k.log", encoding="utf-8", errors="ignore") as f:
    for line in f:
        hits += len(AADHAAR.findall(line))
print(hits)
```

`with` closes the file automatically when the block ends. `errors="ignore"` skips odd bytes that real logs sometimes contain instead of crashing.

4.9 Importing from the repo

`research/aran_core.py` is a module. Any file in the same folder can import its functions.

```
from aran_core import verhoeff_valid, luhn_valid
verhoeff_valid("2363")              # True
luhn_valid("4111111111111111")      # True (a standard test card, not a real one)
```

5. Tests with pytest

A test is a function whose name starts with `test_` and that uses `assert`. If the condition after `assert` is false, pytest reports a failure and shows the values. Tests are how the team knows a change did not break something that worked yesterday.

```python
# test_mask.py
from mask import mask_last4

def test_mask_keeps_last_four():
    assert mask_last4("234123412346") == "XXXXXXXX2346"

def test_mask_keeps_length():
    assert len(mask_last4("4111111111111111")) == 16
```

```
> python -m pytest test_mask.py -v
test_mask.py::test_mask_keeps_last_four PASSED
test_mask.py::test_mask_keeps_length PASSED
```

Why this matters for Aran in particular. Aran's `test_core.py` checks our FF1 encryption against the official sample answers published by NIST. If our output matched only our own expectations, we could have a bug and never know; matching NIST's answers proves the implementation is correct. Write a test for every function you add, including at least one case that should fail (for example, a number with a wrong check digit).

6. A map of the repo

File	What it does	Read it after
`README.md`	Project overview, results, how to run	Now
`research/aran_core.py`	Verhoeff, Luhn, FF1 encryption, checksum-complement tokens, leakage counting	Learn 2 and 3
`research/test_core.py`	Tests proving the core is correct, including NIST vectors	This doc
`research/experiment.py`	Experiments E1 to E4 on real logs	Learn 2 and 4
`research/results.json`	The numbers the experiments produced	Learn 4
`research/fetch_data.sh`	Downloads the public Loghub logs	This doc
`docs/research/aran-research.md`	The full research document	After Learn 2
`docs/learning/`	These seven learning docs	In order

Reading `results.json`. It is JSON, a text format of nested dicts and lists. Load it with Python to explore:

```python
import json
r = json.load(open("results.json"))
print(r.keys())        # the top-level sections, one per experiment
```

7. Practice

Do these in order. Each takes under an hour. Solutions for the first three are at the end; try first.

- [ ] **P1.** Clone the repo, create the venv, and run `python -m pytest test_core.py` until every test passes.
- [ ] **P2.** Write a regex for Indian mobile numbers (10 digits, first digit 6 to 9, optionally preceded by `+91` or `0`). Test it on: `call 9876543210`, `ph +91 8765432109`, `id 1234567890` (should not match), `ts 19876543210` (should not match).
- [ ] **P3.** Write `mask_mobile(s)` that returns `"XXXXXX" + s[-4:]`, plus two tests.
- [ ] **P4.** Run `python experiment.py`, find the number 67481 in `results.json`, and explain to a teammate in two sentences what it counts and why it is so large.
- [ ] **P5.** Make your own branch, commit P2 and P3 with their tests, push, and open a pull request. Ask a teammate to review it.

Solutions.

```
        # P2
        MOBILE = re.compile(r"(?<!\d)(?:\+91[ -]?|0)?[6-9]\d{9}(?!\d)")
        # (?<!\d) means "no digit just before"; (?!\d) means "no digit just after".
        # Together they stop the pattern matching inside a longer number such as
19876543210.

        # P3
        def mask_mobile(s: str) -> str:
            return "XXXXXX" + s[-4:]

        def test_mask_mobile():
            assert mask_mobile("9876543210") == "XXXXXX3210"

        def test_mask_mobile_length():
            assert len(mask_mobile("9876543210")) == 10
```

8. Further learning (free)

- Python: the official tutorial at docs.python.org/3/tutorial, chapters 3 to 5 (about 4 hours).
- Regex: regex101.com for practice; the Python `re` module docs for reference.
- git: "Pro Git" at git-scm.com/book, chapters 1 to 3; learngitbranching.js.org for a visual game.
- Testing: docs.pytest.org, the "Get Started" page.

Reading order for the other docs: Learn 2 (identifiers and check digits), Learn 3 (cryptography), Learn 4 (probability), then Learn 5 (AI and NLP) for Bharadhwaj and Arun, Learn 6 (immune systems) for Inbathamizhan and Aasif, and Learn 7 (law) for everyone.